The Data Refinery — Day 6

Architectural Masterclass: Warehouses, Stored Procedures & Recursive Engines

> 💡 **Daily Motivation:** *"Mastering advanced SQL architecture and data engineering is a marathon, not a sprint. True growth happens when you push yourself that one extra mile, tackle that one extra complex query, and refuse to quit when it gets challenging. Keep your eyes on the finish line. When you cross it, that wide smile will remind you that every single drop of effort built an unbreakable foundation for your career."* 🏃‍♂️💨

🏢 Real-World Scenario: The Friday Night Panic at "InstaDrop"

Imagine you are the Data Engineer at **InstaDrop**, a quick-commerce app. It's Friday night at 8:00 PM with millions ordering groceries simultaneously. The executive dashboard is crawling (taking 4 minutes over 500M rows), your ETL pipeline runs 15 separate queries risking partial failures, and the fraud team needs to trace multi-level referral trees.

1️⃣ Dimensional Modeling: Star Schema vs. Snowflake Schema

When designing analytical data warehouses (BigQuery, Snowflake, Redshift), how you structure tables dictates query speed and storage cost:

- **Star Schema (The Industry Favorite):** Features a central **Fact table** (holding numerical metrics like `order_amount`) surrounded by flat, denormalized **Dimension tables** (`dim_customer`, `dim_store`). Columnar engines love this because it minimizes expensive network shuffles and joins.
- **Snowflake Schema (The Over-Normalized Cousin):** Dimension tables are split into normalized sub-tables (e.g., Items links to SubCategory, which links to Category). While saving minor storage, it forces multi-level JOIN chains that degrade performance in distributed engines.

2️⃣ Materialized Views (The Dashboard Savior)

A **Standard View** is just a query skeleton; every time someone opens the dashboard, aggregations run from scratch. A **Materialized View** stores actual physical results on disk. At InstaDrop, we create materialized views for hourly city revenue so dashboards load instantly with `O(1)` lookup speed.

3️⃣ Stored Procedures (SPs) — The "Pre-Recorded Kitchen Recipe"

> 🍳 **Day-to-Day Analogy:** Running a busy restaurant kitchen means you can't text a chef 4 separate steps individually every time a burger order arrives (slice, grill, add sauce, wrap). Instead, you write down the complete recipe once, name it `MakeSpecialBurger`, and shout `EXEC MakeSpecialBurger` when needed. Similarly, a Stored Procedure bundles SQL statements inside the database server to eliminate chatty network overhead.

4️⃣ Recursive CTEs — The "Office Gossip Chain" or "Family Tree"

> 🌳 **Day-to-Day Analogy:** Tracing office gossip or a family lineage starting from your great-grandfather requires a step-by-step chain reaction: Start with Great-Grandfather John → Ask who his children are → Take those children and ask who *their* children are until you hit the bottom. A **Recursive CTE** combines an initial anchor query, a `UNION ALL`, and a loop querying itself recursively.

💻 Coding Challenge 1: Tracing the Referral Tree (Recursive CTE)

Scenario: InstaDrop wants to track referral generation depth starting from root users (`invited_by IS NULL`).

```
WITH RECURSIVE referral_tree AS (
    -- Anchor Member: Root users
    SELECT user_id, user_name, invited_by, 1 AS generation_level
    FROM users
    WHERE invited_by IS NULL

    UNION ALL

    -- Recursive Member: Find invited users
    SELECT u.user_id, u.user_name, u.invited_by, rt.generation_level + 1
    FROM users u
    INNER JOIN referral_tree rt ON u.invited_by = rt.user_id
)
SELECT * FROM referral_tree ORDER BY generation_level;
```

💻 Coding Challenge 2: Top Delivery Partners

Find the top 2 highest-earning delivery riders in each city using a CTE and a window function:

```
WITH rider_totals AS (
    SELECT
        city, rider_name,
        SUM(payout_amount) AS total_payout,
        ROW_NUMBER() OVER (PARTITION BY city ORDER BY SUM(payout_amount) DESC) as rnk
    FROM rider_payouts
    GROUP BY city, rider_name
)
SELECT city, rider_name, total_payout
FROM rider_totals WHERE rnk <= 2;
```

❓ Interview MCQ Blitz (Q1 – Q5)

Q1: Star vs. Snowflake Schema Performance

Why do data engineers favor Star Schemas in BigQuery/Snowflake?
A) Eliminate redundancy completely. B) Flatten dimensions, reducing joins and network shuffles. C) Snowflake requires more RAM. D) Star schemas cannot use SQL.

Q2: Materialized View Trade-offs

When is a Materialized View a bad idea?
A) When underlying tables receive massive continuous mutations. B) When queries run frequently. C) When daily summaries are needed.

Q3: Recursive CTE Hazard

What hazard must you guard against in graph datasets?
A) UNION ALL usage. B) Circular references causing infinite loops. C) Missing aliases.

Q4: Stored Procedures in MPP Warehouses

Why do row-by-row loops bottleneck performance?
A) MPP engines are built for set-based vectorized execution. B) SQL is unsupported. C) Encryption overhead.

Q5: Fact Table Content

What is stored in a Fact Table?
A) Customer names and addresses. B) Numerical metrics and quantitative values (sales, quantities). C) Auth tokens.

💻 Bonus Coding Challenge 3: 7-Day Rolling Revenue Average

Write a window function query to calculate a 7-day moving average of daily revenue per store to smooth out weekend demand spikes.

```
SELECT
    store_id,
    order_date,
    daily_revenue,
    AVG(daily_revenue) OVER (
        PARTITION BY store_id
        ORDER BY order_date
        ROWS BETWEEN 6 PRECEDING AND CURRENT ROW
    ) AS rolling_7day_avg
FROM daily_store_sales;
```

❓ Bonus MCQ Blitz (Q6 – Q10)

Q6: Slowly Changing Dimensions (SCD Type 2)

How do you track historical changes (e.g., customer address updates) in a dimension table?
A) Overwrite the old record. B) Use surrogate keys, effective start/end dates, and an `is_current` flag. C) Delete the customer.

Q7: Surrogate Keys in Warehouses

Why use surrogate integer keys instead of natural keys in fact tables?
A) Immune to source system changes & faster joins. B) Required by SQL standard. C) Saves network bandwidth.

Q8: Materialized View Refresh Strategies

What distinguishes an incremental refresh from a full refresh?
A) Incremental computes only changed deltas; full recomputes everything. B) Full refresh is faster. C) Incremental deletes base tables.

Q9: Stored Procedure Transactions

Why wrap multi-step ETL updates in transactions (`BEGIN / COMMIT / ROLLBACK`)?
A) Ensures ACID atomicity so partial failures rollback cleanly. B) Speeds up CPU cores. C) Encrypts logs.

Q10: Snowflake Join Complexity

What is a primary drawback of Snowflake schemas?
A) Higher join complexity across normalized sub-dimensions. B) Zero storage savings. C) Inability to store numbers.

> 🔑 **Complete Answer Key:** Q1: B | Q2: A | Q3: B | Q4: A | Q5: B | Q6: B | Q7: A | Q8: A | Q9: A | Q10: A